

Figure 8: Binary Classification task (training and test set performance): Average accuracy (left) and satisfiability (right). Due to the random initializations, accuracy and satisfiability start on average at 0.5 with performance increasing rapidly after a few epochs.

classes A , B or C . This syntax allows one to write statements quantifying over the classes, e.g. $\forall x(\exists l(P(x, l)))$. Since the classes are mutually exclusive, the output layer of the MLP representing $P(x, l)$ will be a `softmax` layer, instead of a `sigmoid` function, to ensure the exclusivity constraint on satisfiability scores¹⁹. The problem can be specified as follows:

Domains:

items, denoting the examples from the Iris flower data set.
labels, denoting the class labels.

Variables:

x_A, x_B, x_C for the positive examples of classes A, B, C .
 x for all examples.
 $\mathbf{D}(x_A) = \mathbf{D}(x_B) = \mathbf{D}(x_C) = \mathbf{D}(x) = \text{items}$.

Constants:

l_A, l_B, l_C , the labels of classes A (Iris setosa), B (Iris virginica), C (Iris versicolor), respectively.
 $\mathbf{D}(l_A) = \mathbf{D}(l_B) = \mathbf{D}(l_C) = \text{labels}$.

Predicates:

$P(x, l)$ denoting the fact that item x is classified as l .
 $\mathbf{D}_{\text{in}}(P) = \text{items, labels}$.

Axioms:

$$\forall x_A P(x_A, l_A) \tag{27}$$

$$\forall x_B P(x_B, l_B) \tag{28}$$

$$\forall x_C P(x_C, l_C) \tag{29}$$

Notice that rules about exclusiveness such as $\forall x(P(x, l_A) \rightarrow (\neg P(x, l_B) \wedge \neg P(x, l_C)))$ are not included since such constraints are already imposed by the grounding of P below, more specifically the `softmax` function.

¹⁹ $\text{softmax}(x) = e^{x_i} / \sum_j e^{x_j}$

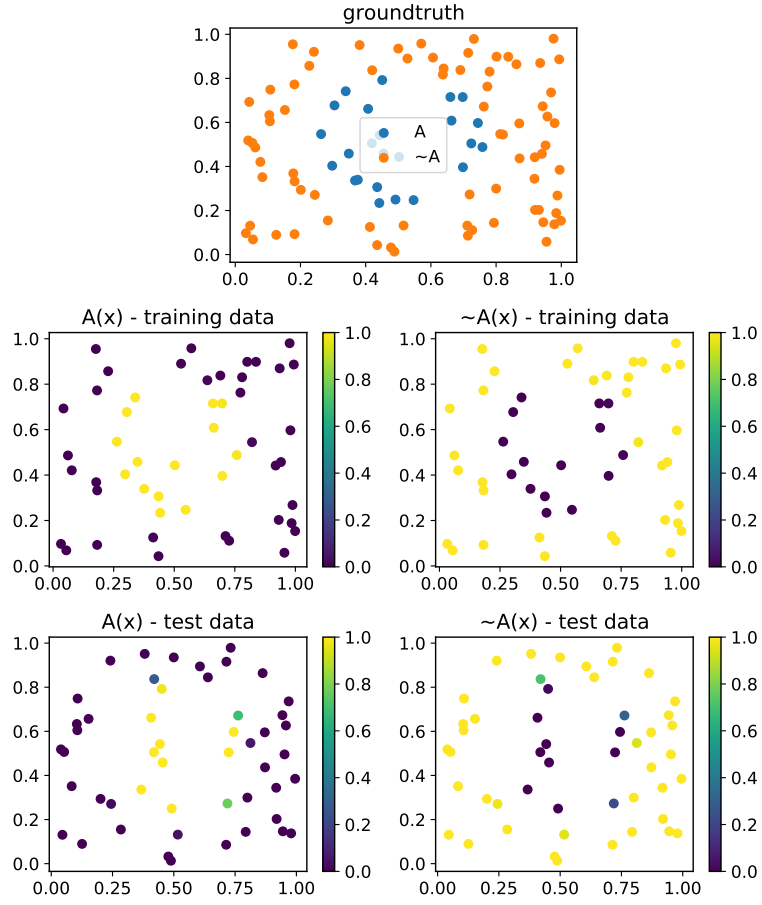


Figure 9: Binary Classification task (querying the trained predicate $A(x)$): It is interesting to see how $A(x)$ could be appropriately named as denoting the inside of the central region shown in the figure, and therefore $\neg A(x)$ represents the outside of the region.

Grounding:

$\mathcal{G}(\text{items}) = \mathbb{R}^4$, items are described by 4 features: the length and the width of the sepals and petals, in centimeters.

$\mathcal{G}(\text{labels}) = \mathbb{N}^3$, we use a one-hot encoding to represent classes.

$\mathcal{G}(x_A) \in \mathbb{R}^{m_1 \times 4}$, that is, $\mathcal{G}(x_A)$ is a sequence of m_1 examples of class A .

$\mathcal{G}(x_B) \in \mathbb{R}^{m_2 \times 4}$, $\mathcal{G}(x_B)$ is a sequence of m_2 examples of class B .

$\mathcal{G}(x_C) \in \mathbb{R}^{m_3 \times 4}$, $\mathcal{G}(x_C)$ is a sequence of m_3 examples of class C .

$\mathcal{G}(x) \in \mathbb{R}^{(m_1+m_2+m_3) \times 4}$, $\mathcal{G}(x)$ is a sequence of all the examples.

$\mathcal{G}(l_A) = [1, 0, 0]$, $\mathcal{G}(l_B) = [0, 1, 0]$, $\mathcal{G}(l_C) = [0, 0, 1]$.

$\mathcal{G}(P \mid \theta) : x, l \mapsto l^\top \cdot \text{softmax}(\text{MLP}_\theta(x))$, where the MLP has three output neurons corresponding to as many classes, and $\cdot$ denotes the dot product as a way of selecting an output for $\mathcal{G}(P \mid \theta)$; multiplying the MLP's output by the one-hot vector $l^\top$ gives the truth degree corresponding to the class denoted by l .

Learning:

The logical operators and connectives are approximated using the stable product configuration with $p = 2$ for A_{pME} . For the formula aggregator, A_{pME} is used also with $p = 2$.

The computational graph of Figure 10 illustrates how $\text{SatAgg}_{\phi \in \mathcal{K}} \mathcal{G}_\theta(\phi)$ is obtained. If U denotes batches sampled from the data set of all examples, the loss function (to minimize) is:

$$L = 1 - \text{SatAgg}_{\phi \in \mathcal{K}} \mathcal{G}_{\theta, x \leftarrow B}(\phi).$$

Figure 11 shows the result of training with the Adam optimizer with batches of 64 examples. Accuracy measures the ratio of examples correctly classified, with example x labeled as $\text{argmax}_l(P(x, l))$.²⁰ Classification accuracy reaches an average value near 1.0 for both the training and test data after some 100 epochs. Satisfaction levels of the Iris flower predictions continue to increase for the rest of the training (500 epochs) to more than 0.8.

It is worth contrasting the choice of using a binary predicate ($P(x, l)$) in this example with the option of using multiple unary predicates ($l_A(x), l_B(x), l_C(x)$), one for each class. Notice how each predicate is normally associated with an output neuron. In the case of the unary predicates, the networks would be disjoint (or modular), whereas weight-sharing takes place with the use of the binary predicate. Since l is instantiated into l_A, l_B, l_C , in practice $P(x, l)$ becomes $P(x, l_A), P(x, l_B), P(x, l_C)$, which is implemented via three output neurons to which a softmax function applies.

4.3. Multi-Class Multi-Label Classification

We now turn to multi-label classification, whereby multiple labels can be assigned to each example. As a first example of the reach of LTNs, we shall see how the previous example can be extended naturally using LTN to account for multiple labels, not always a trivial extension for most ML algorithms. The standard approach to the multi-label problem is to provide explicit negative examples for each class. By contrast, LTN can use background knowledge to relate classes directly

²⁰This is also known as *top-1* accuracy, as proposed in [39]. Cross-entropy results $\sum(t \log(y))$ could have been reported here as is common with the use of softmax, although it is worth noting that, of course, the loss function used by LTN is different.